DB

Theta Join

09/12/25

- different join condition than just equality
- same syntax as equi or natural join

ex.: ... join stocks on
(stores.ID = stocks.store_ID and stock.quantity <=0)

Semi Join

- only attributes of one operand appear in the result → filtering

Top-K

- fraction of a query is kept in a result according to a ranking fct

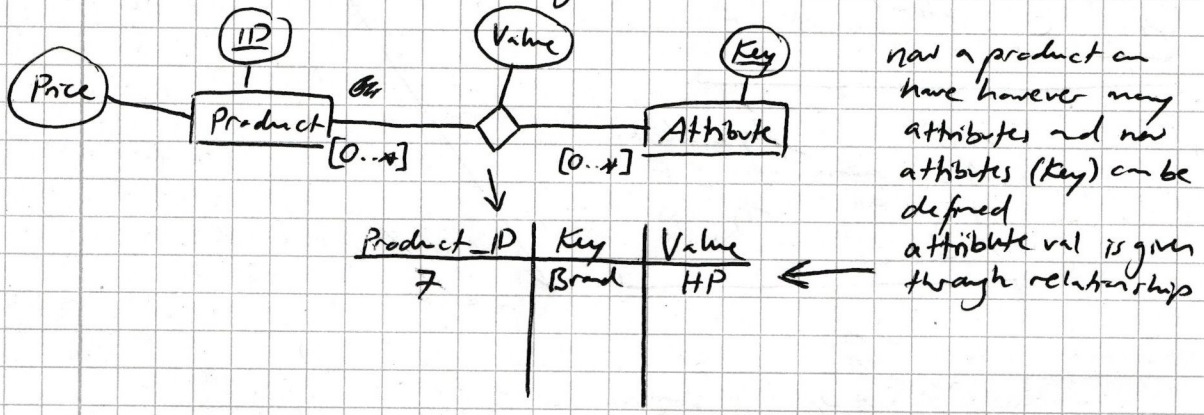
ex.: top(3) → gets top three results of query

select top(3)
[attribute] from [table]

- can be combined with order by to e.g. get latest three orders

Part 2: **Recursion**

Amazon Products : ER Diagram



~~DB~~ DB

Division

- filtering for tuples with given attribute values

10/12/25

> query is done in three parts:
- get all elements we look into
except
- get all possible combinations of filter properties (cross product)
except
- get all actual property combinations (join)

> each part is a select block with the bottom two filtering out tuples that do not fulfill the desired attribute values.

> this selection is then removed from the top query, leaving only tuples that do fulfill the desired attribute values

DB
16/12/25

Chapter 7 – Part 3: PSM

Persistent Stored Modules/Methods

> function can be created and permanently stored in database to reuse for recurring calculations
↳ can be used in queries

> variables: DECLARE @i int = 0;
SET @i = 10;

↳ can be assigned via "=" in select statement

```
select @avg = avg(list_price)
from products
```

↳ can also be assigned via select

```
set @avg = (select AVG(list_price)
            from products)
```

Control flow – if

> if-else > while loop

> case-when-then

```
create function FUNCTION_Name (@PARAMETER TYPE)
returns TYPE
begin
    FUNKTION_BODY (→ return @VARIABLE)
end
```

→ `drop function FUNC_NAME`

Procedure

```
create procedure PROCEDURE_NAME
    @INPUTVAR TYPE,
    @OUTPUTVAR TYPE out
as begin
    DATABASE_CHANGE_BLOCK ]– update DATABASE
                            set ATTRIBUTE_VALUE
                            where CONDITION
end;
```

> used for administration rather than endusers

> procedures can be used to change attribute values in the DB

```
execute PROCEDURE_NAME INPUTVAR, ..., OUTPUTVAR out
```

DB
17/12/25

Cursor – iterator for results of selects

```
declare CURSOR_NAME cursor for
select ATTRIBUTE from TABLE order by ATTRIBUTE asc;
```

→ open CURSOR_NAME (executes select)

→ fetch next from CURSOR_NAME into VARIABLE (of same type and count as select in cursor)

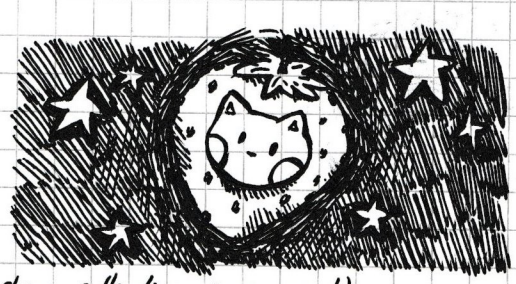
DB
cont'd.

17/12/25

> while loop to iterate over cursor

```
while @@FETCH_STATUS = 0
begin
    // fetch block
end;

deallocate CURSOR_NAME;   (no garbage collection, so manual)
```

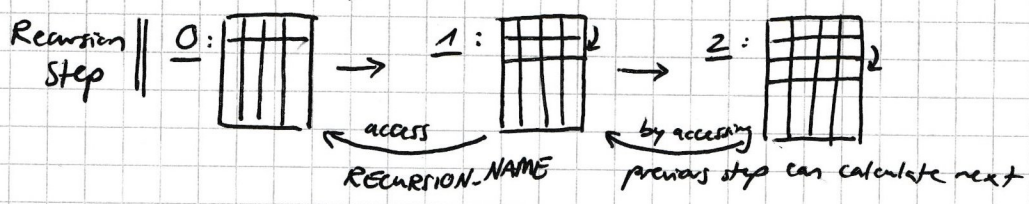


> fetch next (gets next tuple) / prior (gets previous tuple) → ~~cursor~~ cursor must be scrollable, is forward-only by default in TransSQL

Recursion

```
with RECURSION_NAME as (
    // select block   → first recursion step, anchor point
    union all
    // select block   → recursion step, referencing RECURSION_NAME
);                      here ~~with~~ means accessing the ~~final~~ result of the
                        previous recursion step
```

- attribute count and type in select block have to be the same



- each recursion adds to the RECURSION_NAME table which can be accessed with the recursion itself

```
select ATTRIBUTE(S) from RECURSION_NAME;
```

> ~~can~~ can now access attributes from table that recursion created

> recursions can be chained together

```
with RECURSION_1 as (
    // code block
),
RECURSION_2 as (        ← does not have to be a recursion,
    // code block           can also be a division for example
)
```

can use table from previous

> can extend recursion by adding another code block